LEARNING BY DOING

Learn Rust by Building Real Apps

A beginner's guide to Rust — where every single example comes from two services you helped build: a URL shortener and a zero-knowledge pastebin.

★ from variables to async

▸ Rust vs Python

✓ real, shipping code

Topics follow **Rust by Example** · examples are 100% from the repo · no memorization required

Contents

01 Hello, Rust — and why this book exists

★

How to read a book where every example is real

You just helped build **two real web services in Rust**: a **URL shortener** and a **zero-knowledge pastebin**. This book teaches Rust from the ground up — variables, types, ownership, traits, error handling — but it never invents toy examples. **Every snippet is code that actually ships in those two apps.**

That is the whole idea: *learning by doing*. You are not here to memorize syntax. You are here to understand **why** Rust is shaped the way it is, and **what it bought us** when building something that has to be fast, correct, and safe under load.

> **i How each chapter is built**
>
> Every chapter follows the same rhythm: a short idea, a **real code excerpt** from the repo, a plain-English walkthrough, a **Rust vs Python** comparison, and a closing note on **what Rust gave us**. Read it front to back, or jump to a topic.

Here is the program's true entry point — the first thing that runs when the shortener starts. Don't worry about the details yet; just notice it reads like a story: set up logging, load config, build the app, serve it.

url-shortener/src/main.rs

```rust
#[tokio::main]
async fn main() -> ExitCode {
    init_tracing();
    match run().await {
        Ok(()) => ExitCode::SUCCESS,
        Err(err) => {
            tracing::error!(error = %err, "fatal startup error");
            ExitCode::FAILURE
        }
    }
}
```

✓ What Rust gave us here

The function's type says it can fail (it returns a process exit code), and the compiler forced us to handle both the success and error branches of `run()`. Nothing is swept under the rug. We'll unpack `async`, `match`, and `Result` in later chapters.

02 Meet the two apps — a guided tour

What we built, and the shape of the code

Before the language details, here's the lay of the land. Both services share one layout — a **hexagonal (ports & adapters)** architecture — so once you learn one, you can read the other.

The URL shortener

Give it a long URL, it returns a short code and a link like `https://host/Ab3xY7q`. Visiting that code **302-redirects** to the original and counts a hit. It supports custom aliases, expiry (TTL), a host blocklist, per-IP rate limiting, and an optional Redis cache.

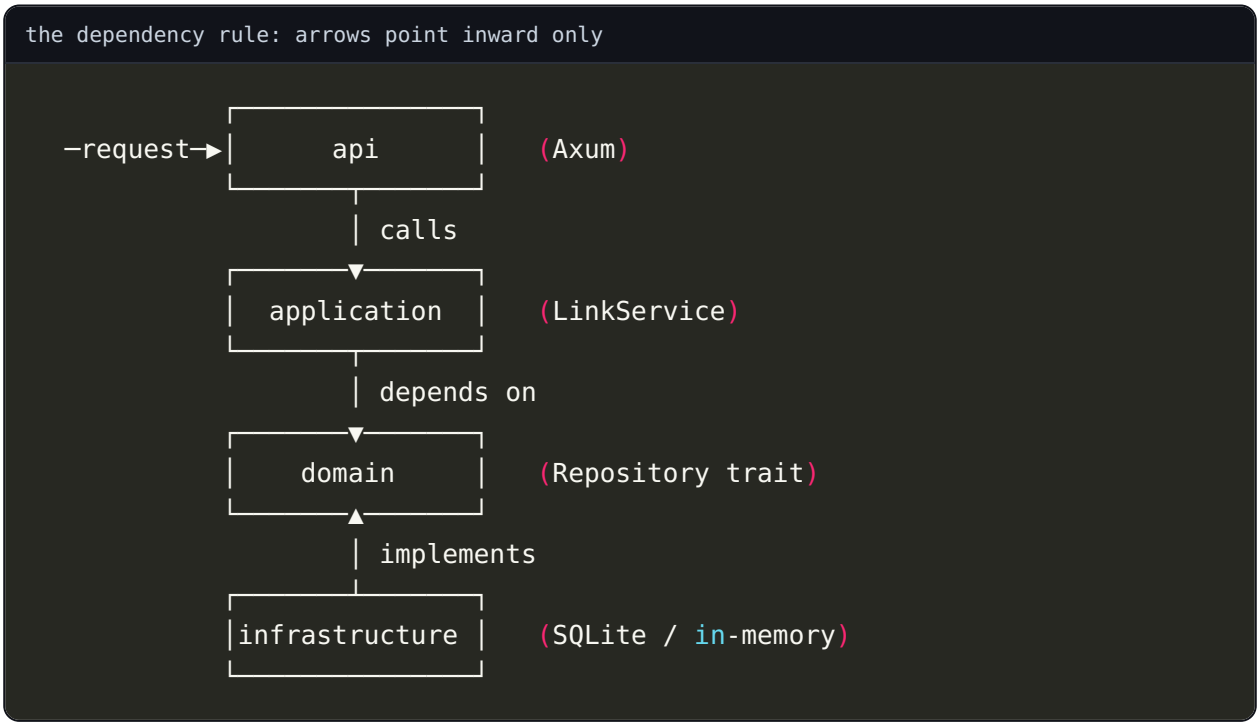
The zero-knowledge pastebin

Store text and get a link back — but the text is **encrypted in the browser** (AES-256-GCM). The key rides in the URL fragment (after `#`) and is never sent to the server, so the server stores only ciphertext. It adds TTL, burn-after-read, an optional password, syntax highlighting, and a QR code.

The layers (same in both)

Code is split into four layers, and dependencies only ever point **inward**:

the dependency rule: arrows point inward only



• **domain** — pure data + rules (no database, no HTTP).

• **application** — the use cases (create, resolve, delete) that orchestrate the domain.

• **infrastructure** — adapters that implement the domain's `Repository` trait (SQLite for real, in-memory for tests).

• **api** — Axum routes/handlers that translate HTTP to use-case calls.

> **⚙ Why this matters for learning**
>
> Each Rust feature in this book lives in a specific layer. Newtypes and traits are in `domain`; `?` and error mapping shine in `application`; async and generics power `api`. The architecture is the map; the language features are the territory.

> **▶ Try it: find your bearings**
>
> Open the repo and list `url-shortener/src/`. You'll see one file per layer: `domain`, `application`, `infrastructure`, `api`, plus `config`, `error`, `main`.

> Keep that folder open as you read — every snippet names its source file.

03 Primitives — numbers, bytes, and booleans

The small fixed-size building blocks

Rust has many number types, and the size is **part of the type**: `u32` is an unsigned 32-bit integer, `i64` a signed 64-bit one, `usize` an integer big enough to index memory. Being explicit is not bureaucracy — it's how Rust guarantees no surprise overflow and zero-cost math. Our configuration is built entirely from primitives:

url-shortener/src/config.rs (excerpt)

```
pub struct Config {
    pub bind_addr: SocketAddr,
    pub max_body_bytes: usize,
    pub database_max_connections: u32,
    pub request_timeout_secs: u64,
    pub rate_limit_rps: u32,
}
```

Lengths and sizes are `usize`; counts are `u32` / `u64`; timestamps are `i64` (Unix seconds). And a boolean is just a boolean — in the pastebin, `one_shot: bool` decides whether a paste self-destructs after one read.

Bytes and characters

When we validate a short code we look at its **bytes** and ask whether every one is an ASCII letter or digit:

url-shortener/src/domain/mod.rs

```
if !value.bytes().all(|b| b.is_ascii_alphanumeric()) {
    return Err(ValidationError::CodeCharset);
}
```

▸ Rust vs Python

Python

```python
MAX = 32                # just a number
n = 5                   # int? float? who knows
ok = c.isalnum()        # works on any unicode
```

Rust

```rust
const CODE_MAX_LEN: usize = 32;   // typed constant
let n: u32 = 5;                   // exact width
let ok = b.is_ascii_alphanumeric();
```

In Python a number is just `int` / `float` decided at runtime. In Rust the width and signedness are fixed at **compile time**, so the CPU does the fastest possible arithmetic and a whole class of bugs simply can't occur.

⚙ Why fixed-size types

A URL shortener counts hits and stores sizes. Choosing `i64` / `usize` means the numbers behave identically on every machine and the compiler can pack structs tightly in memory.

04 Tuples, arrays & slices

Grouping a few values, and borrowing a window into many

A **tuple** bundles a fixed number of values of possibly different types. Our create handler returns one — an HTTP status paired with a JSON body — and Axum knows how to turn that tuple into a response:

url-shortener/src/api/mod.rs

```
Ok((StatusCode::CREATED, Json(body)))
```

A **slice** (`&[T]`) is a borrowed view into a contiguous run of values — you don't own it, you look through it. Our base62 alphabet is a byte-string literal, which is exactly a `&[u8]` slice, and validation borrows the bytes of a string as a slice to inspect them:

url-shortener (application + domain)

```
const ALPHABET: &[u8] =
    b"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";

// elsewhere: look at every byte without copying the string
value.bytes().all(|b| b.is_ascii_alphanumeric())
```

▸ Rust vs Python

Python

```python
ALPHABET = 'ABC...'
# a str; indexing gives another str
for b in s.encode():     # must encode to get bytes
```

Rust

```rust
const ALPHABET: &[u8] = b"ABC...";  // bytes, zero-copy
for b in value.bytes()
{ /* b: u8 */ }
```

Python hides the string-vs-bytes distinction until it bites you (the famous `str` / `bytes` confusion). Rust makes `&str` (text) and `&[u8]` (bytes) different types, so encoding mistakes are caught at compile time.

★ A slice is a pointer + a length

`&[u8]` is just "where the data starts and how long it is." No copy, no allocation — that's why `value.bytes().all(...)` is essentially free.

▶ Try it

In `domain/mod.rs`, find where `value.bytes()` is used. What type does the closure parameter `b` have? (Answer: `u8` — one byte.) Why is that safer than iterating characters here?

05 Custom types — struct and enum

Modeling your domain so wrong states can't exist

A `struct` groups related data. A `Link` in the shortener is exactly the fields that matter, with their types spelled out:

url-shortener/src/domain/mod.rs

```
pub struct Link {
    pub code: ShortCode,
    pub target: TargetUrl,
    pub created_at: i64,
    pub hits: i64,
    pub expires_at: Option<i64>,
}
```

An `enum` says "one of these, never two at once." Our validation errors are an enum — there is no way to accidentally have a code-too-long error that is also a bad-scheme error:

url-shortener/src/domain/mod.rs

```
pub enum ValidationError {
    CodeLength,
    CodeCharset,
    UrlTooLong,
    UrlInvalid,
    UrlScheme,
}
```

▸ Rust vs Python

Python

```python
class Link:
    def __init__(self, code,
target, hits=0, expires_at=None):
        self.code = code
# any type
        self.expires_at =
expires_at

ERR_CODE_LEN = 'code_len'   # a
string, easy to typo
```

Rust

```rust
pub struct Link {
    pub code: ShortCode,
    pub expires_at: Option<i64>,
}

pub enum ValidationError {
CodeLength, /* ... */ }
```

Python models errors as strings or ad-hoc classes; a typo like `'code_lenght'` compiles fine and explodes later. A Rust `enum` is closed and checked — misspell a variant and the build fails instantly.

✓ What Rust gave us here

`Option<i64>` for `expires_at` makes "this link may or may not expire" a fact the compiler tracks. You literally cannot forget to handle the "no expiry" case.

06 Constants & statics

Values fixed at compile time

A `const` is a value baked in at compile time and inlined wherever it's used. Our domain rules are constants, which means they're impossible to mutate and cost nothing at runtime:

url-shortener (domain + application)

```rust
pub const CODE_MIN_LEN: usize = 1;
pub const CODE_MAX_LEN: usize = 32;
pub const URL_MAX_LEN: usize = 2048;

const ALPHABET: &[u8] =
    b"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
const GENERATED_CODE_LEN: usize = 7;
```

Constants name the magic numbers — change the max code length in one place and the whole crate agrees. A `const` has no fixed memory address (it's inlined); a `static` does (one shared instance). For simple values like ours, `const` is the right tool.

▸ Rust vs Python

Python

```python
CODE_MAX_LEN = 32   # a module variable; nothing stops reassignment
CODE_MAX_LEN = 9999 # oops, mutated at runtime
```

Rust

```rust
pub const CODE_MAX_LEN: usize = 32;   // truly constant; reassignment won't compile
```

Python "constants" are just uppercase variables you promise not to change. Rust's `const` is enforced — you cannot reassign it, and its type is fixed.

★ Name your magic numbers

> Every literal in the rules — 1, 32, 2048, 7 — is a named constant. Six months later, `CODE_MAX_LEN` tells you what 32 meant.

07 Variable bindings — let, mut, and shadowing

Immutable by default, mutable on request

In Rust `let` bindings are **immutable by default**. If you want to change a value you must say `let mut`. This flips Python's default and it is a feature: most values never need to change, and the compiler helps keep it that way.

from across the shortener

```
let value = raw.into();           // immutable: never reassigned
let mut rng = rand::thread_rng(); // mutable: we draw from it repeatedly
let bind_addr = config.bind_addr; // a copy we keep after `config` moves
```

Shadowing lets you reuse a name with a new value/type — common when parsing: we take a `String` id and shadow it with a validated `ShortCode`:

url-shortener/src/application/mod.rs

```
let code = ShortCode::parse(code).map_err(|_| ServiceError::NotFound)?;
```

▸ Rust vs Python

Python

```
x = 5
x = 'now a string'    # fine, surprises later
x += 1                # AttributeError at runtime
```

Rust

```
let x = 5;
let x = "now a string"; // shadowing: a new x
// x += 1;  // compile error: types differ
```

Python variables are mutable name-bindings to anything. Rust's default immutability means a value you didn't mark `mut` can never change underneath you — huge for reasoning about concurrent code.

⚙ Why immutable by default

Our request handlers share state across many threads. Immutable-by-default means the compiler can prove most of that state is never mutated, so there's nothing to lock and no data race to worry about.

08 Types: casting and type aliases →

Converting between number types, and naming a scary type

Rust never silently widens or narrows numbers — you convert explicitly with `as`. We turn a random index into a character, and a TTL in seconds into the `i64` our timestamps use:

url-shortener/src/application/mod.rs

```
ALPHABET[rng.gen_range(0..ALPHABET.len())] as char   // u8 -> char
let expires_at = ttl_seconds.map(|secs| created_at.saturating_add(secs as
i64)); // u64 -> i64
```

Notice `saturating_add`: rather than risk overflow, it clamps at the maximum. Explicit casts make these decisions visible. When a type signature gets long and scary, a **type alias** gives it a friendly name — we alias the "any error" box type:

url-shortener/src/domain/mod.rs

```
pub type BoxedError = Box<dyn std::error::Error + Send + Sync + 'static>;
```

▸ Rust vs Python

Python

```
x = 5 / 2
# 2.5 (float!) — silent type
change
y = int('07')    # works, or
throws at runtime
```

Rust

```
let x = 5 / 2;            // 2
(integer division, no surprise)
let n = secs as i64;      //
explicit, intentional cast
```

Python silently turns `/` into float division and coerces types at runtime. Rust keeps integer math integer, and every cross-type conversion is spelled out, so the result type is never a surprise.

⚠ Common mistake

Reaching for `as` to convert a `String` to a number — that doesn't work. `as` is only for primitive numeric/char casts. For text→number you parse (next chapters); for type→type you use `From` / `Into`.

09 Conversion — From, Into, and parse →

Turning one type into another, safely

Rust standardizes conversions through the `From`/`Into` traits. Implement `From<A> for B` and you automatically get `a.into()`. We use this to translate errors as they bubble up through the layers:

url-shortener/src/api/mod.rs

```
impl From<ServiceError> for AppError {
    fn from(err: ServiceError) -> Self {
        match err {
            ServiceError::Validation(v) =>
AppError::Validation(v.to_string()),
            ServiceError::NotFound      => AppError::NotFound,
            ServiceError::Conflict      => AppError::Conflict("short code
already in use".into()),
            ServiceError::Backend(c)    => AppError::Internal(c),
        }
    }
}
```

Because this `From` exists, the `?` operator (see the error-handling chapter) can *automatically* convert an application error into an HTTP error on the way out. We also accept flexible input with `impl Into<String>` so callers can pass a `&str` or a `String` to `parse`.

▸ Rust vs Python

Python

```python
def to_http(err):
    if isinstance(err,
NotFound): return 404
    elif isinstance(err,
Conflict): return 409

# forget a case? silent wrong
status
```

Rust

```rust
impl From<ServiceError> for
AppError { /* match all variants
*/ }
// miss a variant -> compile
error
```

Python's `isinstance` ladders are easy to leave incomplete. Rust's `match` over an enum must be **exhaustive** — add a new error variant and the compiler points at every place that must handle it.

✓ What Rust gave us here

One `From` impl wires error translation into the `?` operator everywhere. Less boilerplate, and impossible to forget a case.

10

Expressions & blocks — almost everything returns a value

if, match, and {} are expressions, not just statements

In Rust, `if`, `match`, and even a block `{ ... }` **evaluate to a value**. The last line of a block (with no semicolon) is its result. Our cache-TTL helper is one expression:

url-shortener/src/application/mod.rs

```
fn cache_ttl_secs(link: &Link) -> u64 {
    match link.expires_at {
        Some(exp) => {
            let remaining = exp - now_unix();
            if remaining <= 0 { 0 } else { (remaining as
u64).min(MAX_CACHE_TTL_SECS) }
        }
        None => MAX_CACHE_TTL_SECS,
    }
}
```

No `return` needed: the `match` is the function body, and each arm is itself an expression. The inner `if/else` produces the number directly. This is why Rust code often has so few `return` statements.

▸ Rust vs Python

Python

```python
if remaining <= 0:
    ttl = 0
else:
    ttl = min(remaining, MAX)
return ttl
```

Rust

```rust
let ttl = if remaining <= 0 {
0 } else { remaining.min(MAX) };
```

In Python `if` is a statement, so you assign inside each branch. In Rust `if` is an expression you can bind directly to a variable — fewer temporaries, and the compiler checks both branches return the same type.

⚠ The semicolon matters

Add a semicolon to that last line and the block returns `()` ("nothing") instead of the value — a classic beginner compile error. No semicolon = "this is the result."

11 Flow of control — match, if let, loops

Branching the compiler checks for completeness

`match` is Rust's pattern-matching workhorse. Validating a URL's scheme is a tiny, total decision:

url-shortener/src/domain/mod.rs

```
match parsed.scheme() {
    "http" | "https" => {}
    _ => return Err(ValidationError::UrlScheme),
}
```

`matches!` is a compact match that returns a bool — perfect for the expiry check, which reads almost like English: *is there an expiry, and is now past it?*

url-shortener/src/domain/mod.rs

```
pub fn is_expired(&self, now: i64) -> bool {
    matches!(self.expires_at, Some(exp) if now >= exp)
}
```

And a bounded `for` loop powers collision-safe code generation: try a few random codes, stop at the first that inserts cleanly.

url-shortener/src/application/mod.rs

```rust
for _ in 0..MAX_GENERATION_ATTEMPTS {
    let code = ShortCode::from_trusted(generate_code(GENERATED_CODE_LEN));
    let link = Link::with_expiry(code, target.clone(), created_at,
expires_at);
    match self.repo.insert(&link).await {
        Ok(())                     => return Ok(link),
        Err(RepoError::Conflict)   => continue,
        Err(RepoError::Backend(c)) => return Err(ServiceError::Backend(c)),
    }
}
```

⚠ A match must cover everything

If we added a third `RepoError` variant, this code would **stop compiling** until we decided what to do with it. The compiler refuses to let a new case slip through unhandled.

12 loop, while & ranges

The three ways Rust repeats work

Rust has `loop` (forever, until you `break`), `while` (while a condition holds), and `for` (over an iterator or range). Our code-generation retry uses a bounded `for` over a **range** `0..N`:

url-shortener/src/application/mod.rs

```
for _ in 0..MAX_GENERATION_ATTEMPTS {
    let code = ShortCode::from_trusted(generate_code(GENERATED_CODE_LEN));
    match self.repo.insert(&link).await {
        Ok(())                      => return Ok(link),
        Err(RepoError::Conflict)    => continue,   // try another code
        Err(RepoError::Backend(c)) => return Err(ServiceError::Backend(c)),
    }
}
Err(ServiceError::Conflict)   // exhausted all attempts (astronomically unlikely)
```

`continue` skips to the next attempt on a collision; `return` leaves the whole function on success or a real error. The range `0..N` is *lazy* — it never builds a list of numbers, it just counts.

▸ Rust vs Python

Python

```
for _ in range(MAX_ATTEMPTS):
    if try_insert(): return code
# range() in py2 built a list;
py3 is lazy like Rust
```

Rust

```
for _ in
0..MAX_GENERATION_ATTEMPTS { /
* ... */ }
```

Both are lazy counters today. The difference: Rust's loop body is checked to handle every `match` arm, so a new error kind can't slip past the retry logic.

▶ Try it

Why is the loop bounded to 5 attempts instead of `loop {}` forever? (Hint: with 62^7 codes, a collision is ~1-in-a-million; five tries is a safety valve, and the final `Err` is essentially unreachable.)

13

Pattern matching, deep — Option & Result combinators

if let, let-else, and the methods that replace boilerplate

Beyond `match`, Rust has ergonomic shortcuts for the common cases. `if let` handles "just the Some/Ok case"; we use it for a cache hit:

url-shortener/src/application/mod.rs

```
if let Some(target) = self.cache.get(code.as_str()).await {
    let _ = self.repo.increment_hits(&code).await;
    return Ok(TargetUrl::from_trusted(target));
}
```

let-else handles "give me the value or bail out now" — the SQLite adapter uses it to return early when a row is missing:

url-shortener/src/infrastructure/sqlite.rs

```
let Some(row) = row else { return Ok(None) };
```

And `Option` / `Result` carry combinator methods so you rarely write a full `match`: `.map_err(|_| ServiceError::NotFound)`, `.ok_or(ServiceError::NotFound)`, `.map(str::to_owned)`, `.unwrap_or`

▸ Rust vs Python

Python

```python
t = cache.get(code)
if t is not None:
    return t
# forget the check -> None leaks downstream
```

Rust

```rust
if let Some(target) = self.cache.get(k).await { return Ok(target); }
```

Python's `if x is not None` is easy to forget, and a stray `None` travels far before crashing. Rust's `if let` / `let-else` make the "absent" branch impossible to skip — the value simply isn't in scope otherwise.

⚠ Common mistake

Reaching for `.unwrap()` to "just get the value." It panics on `None` / `Err`. On a request path we never do this — we use `?`, `ok_or`, or `if let` so failure is handled, not crashed.

14 Functions, methods, and closures

Named logic, attached logic, and inline logic

Free functions stand alone; methods live in an `impl` block and take `&self`. Closures are anonymous functions you pass around — Rust leans on them heavily, especially with iterators. Generating a random code uses a closure inside an iterator chain:

url-shortener/src/application/mod.rs

```
fn generate_code(len: usize) -> String {
    let mut rng = rand::thread_rng();
    (0..len)
        .map(|_| ALPHABET[rng.gen_range(0..ALPHABET.len())] as char)
        .collect()
}
```

`(0..len)` is a range; `.map(|_| ...)` runs a closure for each step; `.collect()` gathers the results into a `String`. The closure `|_|` ignores its argument — we just want `len` random characters.

Closures also appear as error adapters: `.map_err(|_| ServiceError::NotFound)` turns any parse failure into a 404.

▸ Rust vs Python

Python

```
code =
''.join(random.choice(ALPHABET)
for _ in range(n))
```

Rust

```
let code: String = (0..n).map(|
_| ALPHABET[rng.gen_range(0..
62)] as char).collect();
```

The shapes are similar — but Rust's iterator chain compiles to a tight loop with **no allocations per step** and no interpreter overhead, while staying just as readable.

★ Iterators are lazy and fast

`map` / `filter` / `collect` build a pipeline the compiler fuses into one loop — often faster than a hand-written one, with none of Python's per-element overhead.

15 Newtypes — making illegal values unrepresentable

A String that is guaranteed to be a valid code

This is one of Rust's superpowers. Instead of passing raw `String`s around and hoping they're valid, we wrap them in a **newtype** whose only constructor validates. Once you hold a `ShortCode`, it is *provably* valid.

url-shortener/src/domain/mod.rs

```
pub struct ShortCode(String);

impl ShortCode {
    pub fn parse(raw: impl Into<String>) -> Result<Self, ValidationError> {
        let value = raw.into();
        if value.len() < CODE_MIN_LEN || value.len() > CODE_MAX_LEN {
            return Err(ValidationError::CodeLength);
        }
        if !value.bytes().all(|b| b.is_ascii_alphanumeric()) {
            return Err(ValidationError::CodeCharset);
        }
        Ok(Self(value))
    }
    pub fn as_str(&self) -> &str { &self.0 }
}
```

Every function that takes a `ShortCode` can skip re-checking it. The type is the guarantee. The same trick gives us `TargetUrl` (a validated http/https URL) and, in the pastebin, `PasteId` and `Content`.

★ Read the signature

`parse` returns `Result<Self, ValidationError>` — "either a ShortCode or a reason it wasn't one." The caller can't ignore the failure path; that's enforced by the compiler (see the error-handling chapter).

▸ Rust vs Python

Python

```
def make_code(raw):
    return raw            # callers must remember to validate

use_code(make_code('../etc')) # oops, never checked
```

Rust

```
let code = ShortCode::parse(raw)?; // validated or we stopped here
use_code(&code);
guaranteed valid
```

In Python validation is a convention you hope everyone follows. In Rust the *type system* enforces it: you cannot construct a `ShortCode` without going through `parse`.

16 Modules and crates — organizing the code

Namespaces and compilation units

A **crate** is one compilation unit (our shortener is one crate, the pastebin another). Inside a crate, `mod` declarations build a tree of namespaces. The library root wires the layers together:

url-shortener/src/lib.rs

```
#![forbid(unsafe_code)]

pub mod api;
pub mod application;
pub mod cache;
pub mod config;
pub mod domain;
pub mod error;
pub mod infrastructure;
pub mod rate_limit;
```

Each `pub mod` maps to a file or folder. `pub mod api;` tells the compiler “there is a module named `api`, find it in `api.rs` or `api/mod.rs`.” That's the whole file-layout rule: the module tree is declared in code, and the files follow it — there's no implicit “every file is a module” magic like Python's. `pub` controls what *escapes* a module: privacy is the default, so a type or function is visible only inside its own module unless you opt it out with `pub` (or scope it with `pub(crate)` for “public within this crate only”).

Inside the crate you refer to items by **path**: `crate::` starts from the library root, `super::` means the parent module, and `self::` the current one. A `use` line just imports a path so you can write the short name. A powerful move is the **re-export** — `pub use` — which lets an inner item surface at a

friendlier path; that's how `infrastructure` exposes its adapters without revealing its private sub-modules. The first line, `#![forbid(unsafe_code)]`, is a crate-wide attribute that bans `unsafe` anywhere in this crate (see the Unsafe chapter).

▸ Rust vs Python

Python

```
# Python: modules = files, everything public by default
from app.domain import ShortCode   # nothing stops deep imports
```

Rust

```
pub mod domain;            // explicit module tree
// `domain::ShortCode` only if marked `pub`
```

Python exposes everything and relies on a leading-underscore *convention* for "private." Rust makes privacy the default and `pub` a deliberate choice, so your public API is exactly what you intended.

✓ What Rust gave us here

The module tree mirrors our architecture — `domain` → `application` → `infrastructure` → `api` — and the compiler enforces which layer may see which.

17 Cargo and Cargo.toml — build & package tool

Dependencies, builds, and tests in one tool

Cargo is Rust's build system and package manager. `Cargo.toml` declares dependencies and their features; you never hand-manage a virtualenv or a requirements file that drifts out of sync.

url-shortener/Cargo.toml (excerpt)

```
[package]
name = "url-shortener"
edition = "2021"

[dependencies]
axum = "0.7"
tokio = { version = "1", features = ["rt-multi-thread", "macros", "net",
"signal"] }
sqlx = { version = "0.7", default-features = false, features = ["runtime-
tokio", "sqlite", "migrate"] }
serde = { version = "1", features = ["derive"] }
```

Day-to-day you only need a few commands:

the everyday loop

```
cargo run      # build and run the service
cargo test     # compile and run every test
cargo clippy   # lint for common mistakes
cargo fmt      # format the code
```

▸ Rust vs Python

Python

```
pip install -r
requirements.txt   # separate
from running
pytest
# separate test runner
flake8 / black /
mypy             # separate tools
```

Rust

```
cargo build / run / test /
clippy / fmt   # one tool, one
lockfile
```

Python stitches together pip, venv, pytest, black, flake8, mypy. Cargo is all of that in one place, with a `Cargo.lock` pinning exact versions for reproducible builds.

★ Features are compile-time switches

We disabled sqlx's defaults and turned on only `sqlite` + `migrate`.

Unused code is never compiled in — smaller, faster binaries.

18 Attributes — metadata that writes code for you

#[derive], #[tokio::main], #[cfg(test)] and friends

Attributes are annotations in `#[...]`. The most beloved is `#[derive(...)]`, which auto-generates trait implementations so you don't write them by hand:

across the shortener

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub struct ShortCode(String);

#[derive(Debug, Deserialize)]
pub struct CreateLinkRequest {
    pub url: String,
    #[serde(default)]
    pub alias: Option<String>,
}
```

That one `derive` line gives `ShortCode` debug-printing, cloning, equality, and hashability (so it can be a `HashMap` key). `#[serde(default)]` says "if the client omits `alias`, default it to `None`." Other attributes mark the async entry point (`#[tokio::main]`) and test-only code (`#[cfg(test)]`).

▸ Rust vs Python

Python

```python
@dataclass
class CreateReq:
    url: str
    alias: str | None = None
# equality/hash via decorators,
at runtime
```

Rust

```rust
#[derive(Debug, Deserialize)]
struct CreateReq { url: String,
#[serde(default)] alias:
Option<String> }
```

Python's `@dataclass` generates methods at runtime. Rust's `derive` generates them at **compile time** with zero runtime cost, and JSON parsing is type-checked rather than duck-typed.

19

Generics & trait objects — write once, work for many

impl Into, Result<T, E>, and Arc<dyn Trait>

Generics let one piece of code work for many types. `Result<T, E>` is generic over its success and error types. Our `parse` functions accept `impl Into<String>` — any type convertible to a String.

The most important generic in these apps is the **trait object** `Arc<dyn LinkRepository>`: "a shared pointer to *something* that behaves like a repository." Production passes the SQLite version; tests pass an in-memory one; neither the service nor the handlers change.

url-shortener — dependency injection via a trait object

```
pub struct LinkService {
    repo: Arc<dyn LinkRepository>,   // any implementation, chosen at runtime
    cache: Arc<dyn Cache>,
}

pub fn build_app(config: Config, repo: Arc<dyn LinkRepository>) -> Router { /* ... */ }
```

There are two flavors here, and the difference is worth understanding. When you write a generic such as `impl Into<String>` or `fn f<T: Trait>()`, the compiler stamps out a separate, specialized copy of the code for each concrete type you actually use — this is called **monomorphization**, and it's *static dispatch*: no runtime overhead at all, as if you'd hand-written each version. A `dyn Trait` behind a pointer (our `Arc<dyn LinkRepository>`) is the other flavor — *dynamic dispatch* — where the concrete type is decided

at runtime and each method call goes through a tiny lookup table (a “vtable”). The trade is flexibility for a negligible indirection cost.

Rule of thumb: prefer generics when the type is known at compile time and you want maximum speed; reach for `dyn` when you need to *store or swap* different implementations behind one type — which is exactly why our repository and cache are `dyn`, chosen once at startup and never thought about again by the business logic.

▸ Rust vs Python

Python

```python
class Service:
    def __init__(self, repo):
# any object with .insert(), we hope
        self.repo = repo
```

Rust

```rust
struct Service { repo: Arc<dyn LinkRepository> } // must implement the trait, checked
```

Python duck-typing accepts any object and fails at runtime if a method is missing. Rust's `dyn LinkRepository` guarantees at compile time that whatever you inject implements every required method with the right types.

✓ What Rust gave us here

This single abstraction is why our tests need no database, why we could add a Redis cache as just another implementation, and why a future Postgres backend won't touch the business logic.

20 Ownership & borrowing — the heart of Rust §

No garbage collector, no use-after-free

Every value in Rust has a single **owner** — the variable currently responsible for it. When the owner goes out of scope, the value is freed immediately and deterministically, with no garbage collector deciding “later.” This one idea is where Rust's speed *and* its safety both come from.

The part that trips up newcomers is that assigning or passing a value **moves** it. After `let b = a;` (for anything that owns a heap allocation, like a `String`), `a` is no longer usable — ownership transferred to `b`. That feels strict, but it's exactly what guarantees two variables never both think they own — and both try to free — the same memory. Small, cheap types (integers, `bool`, `char`) implement `Copy` and are duplicated instead of moved, so they keep working after assignment.

When you don't want to give ownership away, you **borrow** instead — and borrowing has one rule that powers the whole language: at any given moment a value may have **either** any number of shared references `&T` (read-only) **or** exactly one mutable reference `&mut T` — never both at once. That single constraint is what rules out data races (two threads writing simultaneously) and quieter bugs like mutating a list while you're iterating it. The compiler proves it for every borrow, before the program ever runs.

Methods borrow `&self` and can return a borrowed `&str` pointing *into* the struct — no copy, and the compiler guarantees the string outlives the reference:

url-shortener/src/domain/mod.rs

```rust
pub fn as_str(&self) -> &str {
    &self.0          // a borrow into the owned String; zero-copy, always
valid
}
```

Shared ownership across threads uses `Arc` (atomic reference count). Cloning an `Arc` is cheap — it bumps a counter, it does not copy the data — exactly what we want for state shared by thousands of requests.

▸ Rust vs Python

Python

```python
# Python: a garbage collector
frees objects 'eventually'
data = load()      # freed
whenever the GC decides
# cycles can still leak; GC
pauses happen
```

Rust

```rust
let data = load();          //
owned here
process(&data);             //
borrowed, no copy
// freed exactly when it goes out
of scope
```

Python frees memory with a garbage collector — convenient, but with pauses and cycle leaks. Rust frees memory **deterministically** at scope end, with no GC and no leaks, all proven at compile time.

⚙ Why this matters for a server

No GC pauses means predictable latency under load. Deterministic cleanup means DB connections and locks are released the instant they're done — see the RAII & smart-pointers chapter.

21 Lifetimes — making references provably safe §

The compiler tracks how long a borrow may live

A reference must never outlive the thing it points to. Usually Rust infers the relationships; occasionally you annotate them with a tick name like `'a`. You'll see the anonymous lifetime `'_` in our `Display` impls:

url-shortener/src/domain/mod.rs

```
impl fmt::Display for ShortCode {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str(&self.0)
    }
}
```

The `'_` says "there is a lifetime here; infer it." The borrow checker then proves the formatter reference is valid for exactly as long as `fmt` runs — no dangling pointer is possible.

It helps to read a lifetime as a **region of code**, not a number of seconds: `'a` names “some span over which this reference stays valid,” and the borrow checker's whole job is to prove the value behind the reference lives at least that long. The reason you rarely write lifetimes is **elision** — a handful of defaults the compiler applies to common shapes. The relevant one here: a method that takes `&self` and returns a reference is assumed to return one that borrows from `self`. That's why our `as_str(&self) -> &str` needs no annotation — the rule fills it in. You only reach for an explicit `'a` when a function takes *several* references and the compiler genuinely can't tell which one the result borrows from.

⚠ You can't return a dangling reference

If you tried to return a reference to a value about to be dropped — say, a `&str` into a `String` created inside the function and freed at its end — the compiler rejects it: “returns a value referencing data owned by the current function.” In C that compiles and crashes (or worse, leaks data) at runtime; in Rust it's a friendly error before you ever run the code.

✓ What Rust gave us here

Lifetimes are why our zero-copy `as_str()` is safe: the returned slice can never outlive the `String` it borrows from.

22

RAII & smart pointers — cleanup that can't be forgotten

Arc, Box, MutexGuard, and the connection pool

RAII — *Resource Acquisition Is Initialization* — means a resource is tied to a value, and releasing it is just letting that value drop. Lock guards release on scope exit, and the database connection pool drains when the server shuts down. Our shutdown handler exists precisely so those drops run cleanly:

url-shortener/src/main.rs

```
async fn shutdown_signal() {
    // wait for Ctrl-C or SIGTERM...
    tracing::info!("shutdown signal received");
}
// `axum::serve(...).with_graceful_shutdown(shutdown_signal())`
// lets in-flight requests finish and the pool drain — no leaked handles.
```

The mechanism behind RAII is the `Drop` trait: when any value goes out of scope, Rust runs its `drop` code automatically, in reverse order of creation. You almost never write `Drop` yourself — the library types already do. A `MutexGuard`'s `drop` releases the lock; a `SqlitePool`'s `drop` closes its connections; a `File`'s `drop` closes the handle. There is no `finally`, no `with` block, no manual `close()` to forget — cleanup is wired to the value's lifetime.

Smart pointers are types that own a value and add a capability, while still acting like a reference to it. The three you'll meet constantly: `Box<T>` puts a single value on the heap (we use `Box<dyn Error>` to hold “some

error, type decided at runtime,” which a bare value couldn't because its size isn't known); `Rc<T>` allows several owners within one thread by reference counting; and `Arc<T>` does the same *across* threads using an atomic counter (that's the “A”). Cloning an `Arc` just bumps the count — cheap — and the value is freed when the last clone drops. Each is a smart pointer because it also implements `Drop` to clean up exactly when its last owner goes away.

> **⚙ Why no leaks**
>
> Our docs promise “no leaks.” RAII plus single-ownership make that the default: memory and resources are freed exactly when their owner drops, and we never build the reference cycles (an `Rc` pointing back at itself) that are the one way to leak in safe Rust.

23 Traits — shared behavior, your way ★

Interfaces, but more powerful

A **trait** is a set of methods a type can implement — like an interface. Our persistence **port** is a trait; storage backends implement it:

url-shortener/src/domain/mod.rs

```
#[async_trait::async_trait]
pub trait LinkRepository: Send + Sync + 'static {   // <- supertrait bounds
    async fn insert(&self, link: &Link) -> Result<(), RepoError>;
    async fn get(&self, code: &ShortCode) -> Result<Option<Link>,
RepoError>;

    /// The primitive every backend MUST implement.
    async fn increment_hits_by(&self, code: &ShortCode, n: i64) ->
Result<bool, RepoError>;

    /// A DEFAULT method, written once in terms of the primitive above.
    async fn increment_hits(&self, code: &ShortCode) -> Result<bool,
RepoError> {
        self.increment_hits_by(code, 1).await
    }

    async fn delete(&self, code: &ShortCode) -> Result<bool, RepoError>;
    async fn ping(&self) -> Result<(), RepoError>;
}
```

Two types implement it: `SqliteLinkRepository` (real database) and `InMemoryLinkRepository` (used in tests). The application depends on the **trait**, never on either concrete type — the Dependency Inversion Principle, enforced by the compiler.

Two details in that definition are worth pausing on, because juniors meet them constantly. First, the `: Send + Sync + 'static` after the trait name are

bounds — they say every implementer must *also* be safe to move between threads and not borrow anything short-lived; a type that isn't simply won't be accepted, which is what makes `Arc<dyn LinkRepository>` safe to share across our async tasks. Second, `increment_hits` has a **default body**: backends implement only the primitive `increment_hits_by`, and they inherit the one-at-a-time version for free. That's how a trait stays small for implementers while still offering convenience — the same instinct as a default method on an interface. Traits can also carry **associated types** and be implemented for types you didn't define (a "blanket impl"), which is how we teach Axum to turn our error type into an HTTP response by implementing its `IntoResponse` trait.

▸ Rust vs Python

Python

```
class Repo(ABC):
    @abstractmethod
    def insert(self, link): ...
# forget to implement? error only when called
```

Rust

```
trait LinkRepository { async fn insert(&self, link: &Link) -> Result<(), RepoError>; }
// missing method => compile error
```

Python's abstract base classes catch a missing method only when it's called. Rust refuses to compile a type that claims to implement a trait but doesn't fulfill every method with the exact signature.

24 Interior mutability & the Default trait

Mutating shared data safely, and free constructors

An `Arc` gives shared, *read-only* access — you can't mutate through a `&`. To change shared data you need **interior mutability**: a type that lets you mutate through `&self` while keeping it safe. The family splits by thread: `Cell` and `RefCell` are single-thread (checked at runtime, and `!Sync`, so they can't cross threads); `Mutex`, `RwLock`, and the atomics are the thread-safe ones. Our in-memory cache uses a `Mutex` around a map:

url-shortener/src/cache.rs

```
#[derive(Debug, Default)]
pub struct InMemoryCache {
    entries: Mutex<HashMap<String, String>>,
}
```

That `#[derive(Default)]` is a small gift: it generates a `Default::default()` constructor that builds the value with every field at *its* default — here, an empty map inside an unlocked `Mutex`. Tests call `InMemoryCache::default()` for a fresh cache; `Metrics` derives it for a zeroed counter; `NoOpCache` is a unit struct that's trivially default. (For the deeper rules on which lock to pick — and why we never hold one across `.await` — see the “Shared state” chapter.)

▸ Rust vs Python

Python

```python
class Cache:
    def __init__(self):
        self.entries = {}    # hand-written
        self.lock = Lock()
```

Rust

```rust
#[derive(Debug, Default)]
struct Cache { entries: Mutex<HashMap<String, String>> }
```

Python wires up the empty state and the lock by hand every time. Rust derives the constructor (`Default`) and ties the lock to the data with `Mutex`, so you cannot touch the map without locking.

★ Default is everywhere

Many std types implement `Default` (`0`, `""`, empty `Vec`/`HashMap`, `None`). Deriving it for your struct composes all of those automatically, so an empty-but-valid starting value is one line, not a hand-written constructor.

25 Macros — code that writes code

derive, json!, migrate!, format! and more

Macros run at compile time and expand into real code. You've already used `#[derive]`. Others build values and queries concisely. Returning JSON uses `serde_json::json!`:

url-shortener/src/api/mod.rs

```
async fn health() -> Json<serde_json::Value> {
    Json(serde_json::json!({ "status": "ok" }))
}
```

`sqlx::migrate!("./migrations")` reads our SQL files **at compile time** and embeds them in the binary; `format!` builds strings; `matches!` and `assert_eq!` power our logic and tests. Macros are checked by the compiler — a malformed `json!` won't build.

It's worth knowing macros come in two families. **Declarative** macros (written with `macro_rules!`) match patterns of syntax and expand to code — `vec!`, `format!`, `json!`, and `matches!` are all of this kind. **Procedural** macros are tiny compiler plugins that take your code as tokens and emit new code: `#[derive(Deserialize)]`, the `#[async_trait]` that lets traits have async methods, and `sqlx::migrate!` are procedural. The trailing `!` is how you spot a macro *call* as opposed to a function call. Either way, the expansion is ordinary Rust that the compiler then type-checks — a macro can save you typing, but it can never sneak in code that wouldn't otherwise compile.

⚙ Why compile-time macros

Embedding migrations with a macro means the running service has no external SQL files to lose, and a typo in the path is a build error, not a 3 a.m. production surprise.

⚠ Macros aren't magic strings

Unlike Python's `eval` or string-built SQL, these macros are parsed and type-checked by the compiler, so they can't silently produce invalid code at runtime.

26 Error handling — Result, Option, and ?

Failure is a value you must deal with

Rust has no exceptions for ordinary errors. A function that can fail returns `Result<T, E>`; a value that may be absent is `Option<T>`. The caller **must** acknowledge both outcomes — you can't accidentally ignore an error.

The `?` operator is the glue: "if this is an error, return it (converting types via `From`); otherwise unwrap the success." Watch a whole use-case read top-to-bottom with failure handled at every step:

url-shortener/src/application/mod.rs

```
pub async fn resolve(&self, code: String) -> Result<TargetUrl,
ServiceError> {
    let code = ShortCode::parse(code).map_err(|_| ServiceError::NotFound)?;
    let link = self.repo.get(&code).await?.ok_or(ServiceError::NotFound)?;
    if link.is_expired(now_unix()) {
        let _ = self.repo.delete(&code).await;
        return Err(ServiceError::NotFound);
    }
    self.repo.increment_hits(&code).await?;
    Ok(link.target)
}
```

`?` after `parse`, after `get`, after `increment_hits`: each can fail, and each failure exits early with the right error. `.ok_or(...)` turns "not found" (an `Option`) into an error. No `try/except` wrapping the world; the flow is linear and total.

▸ Rust vs Python

Python

```python
def resolve(code):
    link = repo.get(code)
# None? raises? who knows
    return link.target
# AttributeError if None
```

Rust

```rust
let link =
self.repo.get(&code).await?.ok_or(S
Ok(link.target)
```

Python mixes `None` returns and raised exceptions, and forgetting either crashes at runtime. Rust forces the `None` and the error to be handled *before it compiles* — the dreaded `NoneType has no attribute` cannot happen.

✓ What Rust gave us here

Our handlers never call `unwrap()` on the request path. Every failure is a typed value that flows out as the correct HTTP status. A bug becomes a compile error, not a 500 in production.

27 panic! vs Result — crash, or handle it

Why our request paths never panic

A `panic!` unwinds and aborts the current task — appropriate for truly-impossible situations, never for ordinary failures. `unwrap()` and `expect()` panic if a `Result`/`Option` isn't the happy case. We use them freely in **tests** (a failed unwrap = a failed test) but **never on the request path**, where we use `?` instead.

url-shortener/src/application/mod.rs

```
fn now_unix() -> i64 {
    SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .map(|d| d.as_secs() as i64)
        .unwrap_or(0)        // clock weirdness? clamp — do NOT panic a
live server
}
```

Compare a test, where panicking is exactly what we want:

url-shortener/src/domain/mod.rs (tests)

```
let link = ShortCode::parse("abc").unwrap();  // in a test: panic == test
failure
```

▸ Rust vs Python

Python

```
x = data['key']
# KeyError crashes the request
v = int(s)              #
ValueError crashes the request
```

Rust

```
let v =
self.repo.get(&code).await?.ok_or(S
handled, never panics
```

In Python any unhandled exception bubbles up and can take down a request (or worse). In Rust the request path is built from `Result` + `?`, so failures are values we route to the right HTTP status — the server keeps serving.

⚠ Common mistake

Sprinkling `.unwrap()` to silence the compiler. It compiles, then panics in production. The fix is almost always `?`, `ok_or`, or a `match`.

28 Strings: String vs &str §

Owned text and borrowed text — and why there are two

Rust splits text into two types: `String` (you **own** a growable buffer) and `&str` (you **borrow** a view of someone else's text). Our newtypes own a `String` but hand out a borrowed `&str` — zero-copy:

```
url-shortener (domain + api)
```

```rust
pub fn as_str(&self) -> &str { &self.0 }      // borrow, no allocation

let short_url = format!("{}/{}", base_url.trim_end_matches('/'), code); // build an owned String
```

`as_str` lends a window into the owned string; `format!` builds a brand-new owned `String`. The rule of thumb: take `&str` as input (flexible, cheap), return `String` when you create new text.

▸ Rust vs Python

Python

```python
u = base.rstrip('/') + '/' + code   # one str type; copies a lot
```

Rust

```rust
let u = format!("{}/{}", base.trim_end_matches('/'), code); // explicit owned String
```

Python has one `str` and copies freely behind the scenes. Rust's two types make "do I own this or borrow it?" explicit, so you allocate only when you mean to — and the borrow checker proves a borrowed `&str` never outlives its owner.

★ impl Into<String> = accept either

Our `parse(raw: impl Into<String>)` lets callers pass a `&str` *or* a `String`. Inside, `raw.into()` produces the owned `String` we store.

29 Standard library types — Box, Arc, Vec, HashMap, Option

The everyday containers, and where we use them

Rust's `std` gives you the essentials. We use `Box<dyn Error>` to carry any error across a boundary, `Arc` for shared ownership, `Vec`/`String` for growable data, `Option` for maybe-values, and `HashMap` for the in-memory store:

url-shortener/src/infrastructure/memory.rs

```
// the in-memory store: a plain HashMap owned by ONE task (the actor)
let mut links: HashMap<String, Link> = HashMap::new();
links.insert(code, link);                        // key: String, value: Link
let found: Option<Link> = links.get(&code).cloned();
```

A `HashMap<String, Link>` keyed by code *is* the test database. Here it's owned outright by the repository's task, so it needs no lock; the rate limiter uses the very same container as `HashMap<IpAddr, Bucket>`, but behind a `Mutex` because many request threads touch it at once. Same type, different sharing strategy — your choice, made explicit.

▸ Rust vs Python

Python

```
store = {}              # dict, untyped keys/values
store[code] = link      # any key, any value
```

Rust

```
let mut store: HashMap<String, Link> = HashMap::new();
store.insert(code, link);   // key: String, value: Link
```

Python's `dict` accepts any key and value. Rust's `HashMap` fixes both types, so you can never store the wrong thing or look up with the wrong key type.

★ Option replaces None-juggling

`get` returns `Option<Link>` — present or absent, made explicit. There's no `KeyError` and no silent `None` leaking downstream.

30 Rc, Arc & shared ownership §

When more than one place needs the same value

Single ownership is the default, but sometimes many parts of a program need the same value. `Rc<T>` shares ownership on one thread; `Arc<T>` (*atomic* Rc) shares it across threads by reference counting. A server is multi-threaded, so we use `Arc` for everything shared:

url-shortener/src/lib.rs

```
pub fn build_app(config: Config, repo: Arc<dyn LinkRepository>) -> Router {
    let service = LinkService::with_cache(repo,
config.blocked_hosts.clone(), cache);
    let state = Arc::new(AppState { config, service, rate_limiter,
metrics });
    api::router(state)        // every request handler shares this one Arc
}
```

Cloning an `Arc` bumps an atomic counter — it does **not** copy the data. When the last clone drops, the count hits zero and the value is freed. That's how thousands of concurrent requests can share one `AppState` cheaply and safely: Axum hands each handler a clone of the `Arc`, not a copy of the state.

Two patterns you'll see constantly grow out of this. `Arc<dyn Trait>` — shared ownership of *some* implementation, like our repository — and `Arc<Mutex<T>>` when the shared thing also needs to be mutated (the `Arc` shares it; the `Mutex` guards the writes). The one thing reference counting can't handle on its own is a **cycle** — if two `Arc`s point at each other, their counts never reach zero and the memory leaks. The fix is `Weak<T>`, a non-owning reference that doesn't keep the value alive; we don't need it here because our ownership forms a tree, not a cycle.

▸ Rust vs Python

Python

```
# Python: every object is
implicitly reference-counted + GC
state = AppState()   # shared,
freed 'eventually' by the runtime
```

Rust

```
let state = Arc::new(AppState { /
* ... */ }); // explicit shared
ownership
let s2 =
state.clone();
cheap: +1 to the counter
```

Python reference-counts *everything* automatically (plus a cycle-collecting GC). Rust makes shared ownership opt-in and explicit with `Arc` — you pay the atomic-counter cost only where you actually share, and there's no GC.

⚙ Why Arc and not Rc here

`Arc`'s counter is atomic (thread-safe). Because our state crosses threads, the compiler would reject `Rc` — it isn't `Send`. The type system literally won't let you pick the unsafe option.

31 Collections in action — HashMap operations

insert, get, get_mut, remove — the in-memory store

The in-memory repository is a thin wrapper over `HashMap`, and reading its methods is a tour of the collection API. Each operation maps cleanly to an intent:

url-shortener/src/infrastructure/memory.rs

```
async fn insert(&self, link: &Link) -> Result<(), RepoError> {
    let mut links = self.guard();
    if links.contains_key(link.code.as_str()) {     // already there?
        return Err(RepoError::Conflict);
    }
    links.insert(link.code.as_str().to_owned(), link.clone());
    Ok(())
}

async fn get(&self, code: &ShortCode) -> Result<Option<Link>, RepoError> {
    Ok(self.guard().get(code.as_str()).cloned())     // borrow then clone
out
}

async fn delete(&self, code: &ShortCode) -> Result<bool, RepoError> {
    Ok(self.guard().remove(code.as_str()).is_some()) // removed? true/false
}
```

`contains_key` checks membership; `insert` adds; `get` returns an `Option` reference we `.cloned()` to hand out an owned copy; `remove` returns the old value as an `Option`, and `.is_some()` turns that into "did anything get deleted?"

▸ Rust vs Python

Python

```
if code in store: raise Conflict
store[code] = link
val = store.get(code)          #
None if absent
del store[code]                #
KeyError if absent!
```

Rust

```
if links.contains_key(k) {
return Err(Conflict); }
links.insert(k, link);
let val =
links.get(k).cloned();   //
Option, no surprise
links.remove(k);
returns Option, never panics
```

Python's `del store[k]` throws if the key is missing; Rust's `remove` returns an `Option` you can inspect. Every absence is a value to handle, never a runtime crash.

★ get_mut for in-place edits

To bump a hit counter we use `get_mut` — a mutable borrow into the map — then `link.hits += 1`. The borrow checker guarantees no one else touches that entry meanwhile.

32

Iterators in depth

map, filter, all, any, collect — pipelines that compile to tight loops

Iterators are Rust's most-loved tool. They're **lazy** (nothing happens until you consume them) and the compiler fuses a chain into a single efficient loop. We parse a comma-separated blocklist from config with a full pipeline:

url-shortener/src/config.rs

```
let blocked_hosts = env_or("BLOCKED_HOSTS", "")
    .split(',')                       // iterator of &str pieces
    .map(|s| s.trim().to_ascii_lowercase())  // normalize each
    .filter(|s| !s.is_empty())        // drop blanks
    .collect();                       // gather into a Vec<String>
```

And we ask questions of collections without writing loops — "is the host blocked?" is an `any`, "are all bytes alphanumeric?" is an `all`:

url-shortener/src/application/mod.rs

```
self.blocked_hosts.iter().any(|b| host == *b || host.ends_with(&format!(".
{b}")))
```

▸ Rust vs Python

Python

```python
blocked = [s.strip().lower() for
s in raw.split(',') if s.strip()]
hit = any(host == b or
host.endswith('.'+b) for b in
blocked)
```

Rust

```rust
let blocked: Vec<String> =
raw.split(',').map(|s|
s.trim().to_lowercase()).filter(|
s| !s.is_empty()).collect();
let hit = blocked.iter().any(|b|
host == *b ||
host.ends_with(&format!(".
{b}")));
```

They read almost identically — but Rust's chain has no per-element interpreter overhead, allocates only at the final `collect`, and is type-checked end to end.

▶ Try it

Rewrite the blocklist parse as a plain `for` loop in your head. Notice how the iterator version says *what* you want (split, normalize, drop blanks) instead of *how* to loop — that's the appeal.

33 Debug & Display — printing values

Two ways to turn a value into text

Rust separates **developer** printing from **user** printing. `#[derive(Debug)]` gives you `{:?}` formatting for logs and tests; implementing `Display` gives you `{}` for human-facing text. Our newtypes derive `Debug` and implement `Display`:

url-shortener/src/domain/mod.rs

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub struct ShortCode(String);

impl fmt::Display for ShortCode {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str(&self.0)
    }
}
```

Our error enums get their `Display` text from the `#[error("...")]` attribute (via the `thiserror` crate), and tracing logs use `%err` (Display) or `?err` (Debug). So an error has both a tidy message and a detailed debug form.

▸ Rust vs Python

Python

```python
class ShortCode:
    def __repr__(self): ...    # debug-ish
    def __str__(self): ...     # user-ish — easy to forget one
```

Rust

```rust
#[derive(Debug)]            // {:?} for free
impl fmt::Display for ShortCode { /* {} */ }
```

Python has `__repr__` / `__str__` you write by hand. Rust derives `Debug` automatically and lets you opt into `Display` deliberately — so debug output always exists, and user output is intentional.

▶ Try it

Add `tracing::debug!(?link, "resolved")` somewhere and note the `?` sigil — it asks for the `Debug` form. Switch to `%` and the compiler will demand a `Display` impl.

34 Concurrency & async — how it actually works ⚡

What a Future is, what Tokio does, and the one rule

Start with the problem. A web server spends almost all its time *waiting* — for the database to answer, for bytes to arrive over the network. If a thread just sits there blocked while it waits, it's doing nothing useful. The simple fix — one OS thread per request — works until it doesn't: each thread costs memory and the operating system has to keep switching between them, so a few thousand slow connections bring the box to its knees.

Async flips this around: let *one* thread juggle thousands of waiting operations. The trick is that an `async fn` doesn't run when you call it — it returns a **Future**, a value that represents “a computation that isn't finished yet.” You then `.await` it, which means: *pause right here until this is ready, and meanwhile let the thread go do other work.* Here's a real handler:

url-shortener/src/api/mod.rs

```
async fn redirect(
    State(state): State<Arc<AppState>>,
    Path(code): Path<String>,
) -> Result<Response, AppError> {
    let target = state.service.resolve(code).await?;   // <-- pause here; free the thread
    // ...build the 302 redirect once the answer is back...
}
```

While this request is parked at `.await` waiting on the database, the very same thread is off serving other requests. Nothing is blocked. That is the whole idea.

Who runs the futures? The runtime.

A Future is **lazy** — it does nothing on its own; something has to keep poking it to make progress. That something is the async **runtime**, and in our apps it's **Tokio**. The `#[tokio::main]` attribute builds the runtime and runs your `main` on top of it:

url-shortener/src/main.rs

```
#[tokio::main]                          // builds the runtime, runs main on
it
async fn main() -> ExitCode { /* ... */ }

async fn run() -> Result<(), BoxError> {
    let listener = TcpListener::bind(bind_addr).await?;   // await: wait
for the socket
    axum::serve(listener,
app)                                // serve forever, yielding
        .with_graceful_shutdown(shutdown_signal())        //   between
every request
        .await?;
    Ok(())
}
```

Tasks vs threads

When you *do* want things running independently, you spawn a **task** with `tokio::spawn`. A task is like a thread but far cheaper — the runtime multiplexes thousands of them onto a small pool of OS threads. Our in-memory store's owning task and the hit-batcher are both spawned tasks:

url-shortener/src/infrastructure/memory.rs

```
let (tx, rx) = mpsc::unbounded_channel();
tokio::spawn(run(rx));   // a task: like a thread, but lightweight — thousands are fine
```

⚠ The one rule: never block the thread

Because many tasks share one thread, a single task that *blocks* freezes all the others stuck on that thread. So in async code: no `std::thread::sleep` (use `tokio::time::sleep(...).await`), no long CPU-bound loops (offload them with `spawn_blocking`), and — the reason it keeps coming up — **never hold a `Mutex` across an `.await`**. Hold the lock, do the quick work, drop it, *then* await.

▸ Rust vs Python

Python

```
# Python asyncio: same shapes
async def redirect(code):
    target = await resolve(code)
    ...
asyncio.run(main())
```

Rust

```
// Rust + Tokio: same shapes, but the
// runtime spreads tasks across real threads.
async fn redirect(code: String) -> ... {
    let target = resolve(code).await?;
}
#[tokio::main] async fn main() { }
```

If you've used Python's asyncio, the `async` / `await` shapes are identical. The difference: Python's event loop runs on one core (the GIL), while Tokio schedules tasks across many OS threads — so Rust gets real parallelism, and the compiler still proves it's data-race-free.

✓ What Rust gave us here

Async with no garbage collector and no hidden cost: each `async fn` compiles down to a plain state machine. We serve thousands of

> concurrent requests on a few threads, and `Send` / `Sync` still guarantee the multi-threaded runtime can't introduce a data race.

35 Time & background work ⚡

Clocks, durations, and doing work off the hot path

Two clocks matter. `SystemTime` is wall-clock time (for timestamps); `Instant` is a monotonic stopwatch (for measuring elapsed time). We use `SystemTime` for a paste/link's `created_at`:

url-shortener/src/application/mod.rs

```rust
fn now_unix() -> i64 {
    SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .map(|d| d.as_secs() as i64)
        .unwrap_or(0)   // clock before 1970? clamp instead of panic
}
```

The rate limiter uses `Instant` + `Duration` to refill a token bucket, and the SQLite pool uses a `Duration` busy-timeout. Both are typed: you can't accidentally add "5 seconds" to a wall-clock timestamp.

url-shortener (rate_limit + sqlite)

```rust
let elapsed = now.duration_since(bucket.last).as_secs_f64();   // Instant
math
.busy_timeout(Duration::from_secs(5))                          // a typed
duration
```

⚙ Off the hot path

Our scaling notes call for counting redirect hits in the background (batch them, or use Redis), so the write never slows the redirect. Rust's channels (`tokio::sync::mpsc`) are the idiomatic tool: handlers send a message, one task drains and writes — lock-free and typed.

▸ Rust vs Python

Python

```python
import time
now = int(time.time())      # one fuzzy 'time' concept
```

Rust

```rust
let now = SystemTime::now();   // wall clock
let t = Instant::now();        // monotonic stopwatch — different type
```

Python gives you one `time.time()` and you hope you used the right clock. Rust separates wall-clock from monotonic time into different types, so a duration measured with a stopwatch can't be confused with a calendar timestamp.

36 Send + Sync — the invisible thread-safety check §

Two marker traits that make concurrency fearless

Two special traits decide what may cross threads: `Send` (safe to *move* to another thread) and `Sync` (safe to *share* by reference). You rarely write them — the compiler figures them out — but you'll see them as bounds on our repository trait:

url-shortener/src/domain/mod.rs

```
pub trait LinkRepository: Send + Sync + 'static {
    async fn insert(&self, link: &Link) -> Result<(), RepoError>;
    // ...
}
```

That bound says: "any repository must be safe to share across the many threads serving requests." It's why we store it as `Arc` (which is `Send`/`Sync`) and not `Rc` (which isn't). Pick the wrong one and the build fails — the danger is caught at compile time.

What makes these **marker traits** unusual is that they have no methods and you almost never implement them by hand — the compiler adds them automatically whenever it's safe. The rule is compositional: a type is `Send` if all of its parts are `Send`, and `Sync` if all of its parts are `Sync`. That's why the guarantee is trustworthy — it's derived from the pieces all the way down. A few cases worth memorizing: `Rc` is neither (its counter isn't atomic), which is exactly why it can't cross threads; `RefCell` is `Send` but not `Sync` (its borrow flag isn't atomic); and wrapping data in a `Mutex` or sharing it via `Arc` is what *adds* `Sync` back. So when the compiler says a

type is “not `Send`,” it isn't being fussy — it found a genuinely thread-unsafe piece inside, and it found it before you shipped.

▸ Rust vs Python

Python

```
# Python: the GIL serializes
threads; 'thread-safe' is a
runtime hope
# share whatever you like; race
conditions surface as flaky bugs
```

Rust

```
trait LinkRepository: Send +
Sync { /* ... */ }
// share across threads only if
the compiler proves it's safe
```

Python's GIL means real parallelism is limited and "is this safe to share?" is answered by hope and testing. Rust answers it at compile time with `Send` / `Sync`: data races are a *compile error*, not a 2 a.m. incident.

✓ What Rust gave us here

This is the mechanism behind "fearless concurrency." We share one `AppState` across thousands of requests and the compiler guarantees there's no data race — no manual auditing required.

37 Shared state — Mutex, RwLock, and the rules

Mutating data that many tasks hold at once

Thousands of requests hit one process at once, and some need to touch the same data. The rate limiter is the clearest case: every request looks up its IP's token bucket and mutates it. But the limiter is shared as `&self` behind an `Arc` — so how do you mutate through a *shared* reference?

url-shortener/src/rate_limit.rs

```
pub struct RateLimiter {
    capacity: f64,
    refill_per_sec: f64,
    buckets: Mutex<HashMap<IpAddr, Bucket>>,   // interior mutability
}

impl RateLimiter {
    pub fn check(&self, ip: IpAddr) -> bool {          // &self, not &mut self
        if self.refill_per_sec <= 0.0 { return true; }
        let now = Instant::now();
        let mut buckets = self.buckets
            .lock()                                     // -> MutexGuard: unlocks on drop
            .unwrap_or_else(|e| e.into_inner());        // recover if a holder panicked
        let bucket = buckets
            .entry(ip)
            .or_insert(Bucket { tokens: self.capacity, last: now });
        // ...refill, then try to spend one token...
        bucket.tokens >= 1.0
    }
}
```

That's **interior mutability**: a `Mutex` lets you mutate the map through `&self`, and the type system makes sharing it across threads safe (`Mutex` is

`Sync`; `RefCell` isn't, so it wouldn't even compile behind an `Arc`). The `.lock()` call returns a `MutexGuard` — an RAII handle that *unlocks automatically when it drops*, so you can never forget to release it.

The two rules that keep it fast and safe

Hold the lock for the shortest time possible — a few synchronous lines, **never across an `.await`** (that would freeze the runtime and break `Send`). And prefer the cheapest tool that fits: a read-heavy map wants `RwLock` (many readers in parallel, exclusive writers); a single counter wants an atomic (next chapter); and data that many tasks only *read* needs no lock at all — just `Arc`.

> **⚠ "Lock-free is always better" is a myth**
>
> Lock-free is a precise progress guarantee, not the absence of the word `Mutex` — and it is *harder* to get right and often slower under contention. An uncontended `Mutex` lock is a few nanoseconds. Reach for hand-rolled lock-free code only after you have measured a lock as the bottleneck; for a short critical section a `Mutex` is the correct, honest choice.

▸ Rust vs Python

Python

```
# Python: the GIL quietly
serializes this,
# so you just mutate and hope.
buckets[ip].tokens -= 1
```

Rust

```
// Rust makes you name the
synchronization
// and checks it at compile time.
let mut b =
self.buckets.lock().unwrap();
b.get_mut(&ip).unwrap().tokens -
= 1.0;
```

Python hides shared-state hazards behind the GIL; Rust forces them into the open and proves they are handled.

✓ What Rust gave us here

Data races on shared state are a *compile error*, not a 3 a.m. page. The guard guarantees the lock is released; `Send` / `Sync` guarantee the sharing is sound.

38 Atomics — genuinely lock-free counting ⚡

When the shared thing is just a number

Sometimes shared state is a single number. Our `/metrics` endpoint counts redirects served across every worker thread. A `Mutex` would work, but it is overkill — an **atomic** integer can be bumped by many threads at once with no lock:

url-shortener/src/metrics.rs

```
use std::sync::atomic::{AtomicU64, Ordering};

#[derive(Debug, Default)]
pub struct Metrics {
    redirects: AtomicU64,
}

impl Metrics {
    pub fn record_redirect(&self) {                // &self: no lock, no &mut
        self.redirects.fetch_add(1, Ordering::Relaxed);
    }
    pub fn redirects(&self) -> u64 {
        self.redirects.load(Ordering::Relaxed)
    }
}
```

`fetch_add` reads, adds, and writes as one indivisible hardware step, so two threads incrementing at the same instant can never lose a count — and because no lock is taken, no thread ever blocks another. That is what *lock-free* actually means.

Why <code>Relaxed</code>?

Every atomic op carries a memory *ordering* describing how it synchronizes with *other* memory. Our counter guards nothing else — we only need each bump to be atomic and the total eventually right — so `Relaxed`, the cheapest ordering, is exactly correct. You reach for `Acquire`/`Release` only when an atomic gates access to other data (like a flag that publishes a buffer).

★ This is the one place lock-free is unambiguous

A single integer, safe, no `unsafe`, no dependency. Atomics are the sweet spot of lock-free programming; whole lock-free *data structures* are a different and much harder game, usually needing `unsafe` or a vetted crate.

▸ Rust vs Python

Python

```python
# Python: count += 1 is NOT atomic,
# so you need a Lock around it.
with lock:
    self.redirects += 1
```

Rust

```rust
// Rust: one atomic instruction, no lock.
self.redirects.fetch_add(1, Ordering::Relaxed);
```

What needs an explicit lock in Python is a single lock-free instruction in Rust.

✓ What Rust gave us here

A correct, contention-free counter shared across every thread, in two lines, with the cost of a lock removed entirely.

39 Message passing — channels and the actor pattern

Share memory by communicating

There is a second way to handle shared state, opposite in spirit to a lock: instead of many threads reaching into one piece of data, give the data to **one** task and have everyone else *send it messages*. Channels make this natural. Our in-memory repository is built this way — one task owns the map, and there is no lock anywhere:

url-shortener/src/infrastructure/memory.rs

```
enum Cmd {
    Get    { code: String, reply: oneshot::Sender<Option<Link>> },
    Insert { link: Link,   reply: oneshot::Sender<bool> },
    // IncrementBy, Delete, Ping ...
}

pub struct InMemoryLinkRepository {
    tx: mpsc::UnboundedSender<Cmd>,            // just a sender — no Mutex
}

async fn run(mut rx: mpsc::UnboundedReceiver<Cmd>) {
    let mut links: HashMap<String, Link> = HashMap::new();   // owned by
ONE task
    while let Some(cmd) = rx.recv().await {
        match cmd {
            Cmd::Get { code, reply } => { let _ =
reply.send(links.get(&code).cloned()); }
            // ...one arm per command...
        }
    }
}
```

A caller sends a command plus a one-shot *reply* channel, then awaits the answer:

url-shortener/src/infrastructure/memory.rs

```
async fn get(&self, code: &ShortCode) -> Result<Option<Link>, RepoError> {
    let (reply, rx) = oneshot::channel();
    self.tx.send(Cmd::Get { code: code.as_str().to_owned(), reply })?;
    rx.await.map_err(|_| actor_stopped())
}
```

Because one task owns the map, it mutates with a plain `&mut` — no lock, no guard, no poisoning — and data races are impossible *by construction*. That is the **actor pattern**.

Channels also shine for fire-and-forget work

Counting a hit on every redirect shouldn't make the redirect wait on a database write, so the hot path just *enqueues* the code and a background task batches the writes — driven by a `select!` over the channel and a timer:

url-shortener/src/hits.rs

```
tokio::select! {
    maybe = rx.recv() => match maybe {
        Some(Msg::Hit(code))  => { *pending.entry(code).or_insert(0) +=
1; }
        Some(Msg::Flush(ack)) => { flush(&repo, &mut pending).await; let _
= ack.send(()); }
        None => { flush(&repo, &mut pending).await; return; }   // drained
on shutdown
    },
    _ = ticker.tick() => flush(&repo, &mut pending).await,       // ...or
every few seconds
}
```

> **⚙ When to send messages vs. take a lock**
>
> Channels win when work is **fire-and-forget**, **batchable**, or genuinely sequential — like the hit counter. They cost something too: every call becomes a round-trip serialized on one task, so for a hot *read* path a

lock (or no lock) is faster. We use an actor for the in-memory store and a channel for hit-batching; the rate limiter keeps its `Mutex`. Match the tool to the access pattern.

▸ Rust vs Python

Python

```python
# Python: a queue + a worker thread.
q.put(('get', code))
# worker: while True: cmd = q.get() ...
```

Rust

```rust
// Rust: typed messages, a typed reply,
// the compiler enforces both.
self.tx.send(Cmd::Get { code, reply })?;
let value = rx.await?;
```

Same idea as a Python queue + worker, but the message shapes and replies are checked at compile time.

✓ What Rust gave us here

Two safe ways to share — locks and messages — and a type system that makes the actor's ownership and every message's shape impossible to get wrong.

40 Testing — unit, integration, and a test double ✓

Tests that compile with the code and need no database

Tests live next to the code. Unit tests sit in a `#[cfg(test)]` module (compiled only for tests); integration tests live in `tests/` and exercise the whole app. Here's a real unit test of the expiry boundary:

url-shortener/src/domain/mod.rs

```rust
#[test]
fn is_expired_respects_the_boundary() {
    let link = Link::with_expiry(
        ShortCode::parse("abc").unwrap(),
        TargetUrl::parse("https://example.com").unwrap(),
        1_700_000_000,
        Some(1_700_000_100),
    );
    assert!(!link.is_expired(1_700_000_099));
    assert!(link.is_expired(1_700_000_100)); // at expiry == expired
}
```

Because storage is a trait, integration tests inject the in-memory repository and run the real Axum app with **no database at all** — fast and deterministic. `#[tokio::test]` spins up an async runtime per test.

▸ Rust vs Python

Python

```python
# pytest: separate folder,
separate tool, separate config
def test_expiry():
    assert not link.is_expired(t)
```

Rust

```rust
#[test]
fn
is_expired_respects_the_boundary()
{ assert!(!link.is_expired(t)); }
// `cargo test` finds and runs it
```

Python testing is a separate ecosystem you bolt on. In Rust tests ship *inside the crate*, compile with it (so they can't rot), and run with `cargo test`. Our suite is ~85 tests across both services.

✓ What Rust gave us here

The trait-based design means most logic is testable without I/O, and tests are first-class — no "it compiles but the tests were never updated" drift.

41 Unsafe — the escape hatch we refused

Why both services say #![forbid(unsafe_code)]

Rust lets you opt out of its guarantees inside an `unsafe` block for low-level work. We did the opposite: the very first line of each crate **bans it entirely**.

top of url-shortener/src/lib.rs and main.rs

```
#![forbid(unsafe_code)]
```

This is a promise the compiler enforces: not one line of `unsafe` can exist in our code. For a network service handling untrusted input, that rules out the entire category of memory-corruption vulnerabilities (buffer overflows, use-after-free) that plague C and C++ servers.

▸ Rust vs Python

Python

```
# Python is memory-safe too, but
via a GC + the GIL,
# and you can't drop to the metal
when you need to
```

Rust

```
#![forbid(unsafe_code)]   //
memory-safe AND no GC, by choice
```

Python is memory-safe via a garbage collector and interpreter. Rust gives the same safety **without** the runtime cost, and the option (which we declined) to go low-level when a hotspot truly needs it.

✓ What Rust gave us here

> A blanket guarantee, checked on every build, that these services contain zero memory-unsafe code — assurance you cannot get from C, and get for free here.

42 serde — typed JSON, in and out

Parsing and producing JSON that the compiler checks

Web APIs speak JSON. In Rust, `serde` derives the conversion between your structs and JSON automatically — and it's **type-checked**. Our request and response bodies are plain structs with a derive:

url-shortener/src/api/mod.rs

```
#[derive(Debug, Deserialize)]
pub struct CreateLinkRequest {
    pub url: String,
    #[serde(default)]
    pub alias: Option<String>,
}

#[derive(Debug, Serialize)]
pub struct CreateLinkResponse {
    pub code: String,
    pub short_url: String,
    pub expires_at: Option<i64>,
}
```

`Deserialize` turns an incoming JSON body into a `CreateLinkRequest` (rejecting anything malformed); `Serialize` turns our response struct back into JSON. `#[serde(default)]` means a missing `alias` becomes `None` instead of an error.

▸ Rust vs Python

Python

```
data = request.json()          # a dict; keys may or may not exist
alias = data.get('alias')      # KeyError if you forget .get
```

Rust

```
let req: CreateLinkRequest = /* parsed by Axum's Json extractor */;
let alias = req.alias;         // typed Option<String>, guaranteed shape
```

Python hands you an untyped `dict` and you probe keys at runtime. serde parses straight into a typed struct: a missing required field or a wrong type is rejected before your handler runs — no `KeyError` surprises.

▶ Try it

In `api/mod.rs`, find `CreateLinkResponse`. Add a field and watch the compiler require you to set it everywhere the response is built. That's serde + the type system keeping your API honest.

43 Axum — routes, extractors & handlers

Turning HTTP into ordinary function calls

Axum maps URLs to `async` functions. The router lists paths and methods; each handler declares, in its parameters, exactly what it needs from the request — Axum **extracts** those for you:

url-shortener/src/api/mod.rs

```
Router::new()
    .route("/health", get(health))
    .route("/api/links", post(create_link))
    .route("/api/links/:code", get(get_link).delete(delete_link))
    .route("/:code", get(redirect))
```

url-shortener/src/api/mod.rs

```
async fn create_link(
    State(state): State<Arc<AppState>>,   // shared app state
    Json(req): Json<CreateLinkRequest>,   // parsed JSON body
) -> Result<(StatusCode, Json<CreateLinkResponse>), AppError> {
    let link = state.service.create(req.url, req.alias,
req.ttl_seconds).await?;
    Ok((StatusCode::CREATED, Json(CreateLinkResponse::from_link(&link,
&state.config.public_base_url))))
}
```

The parameter types *are* the request contract: `State` for shared state, `Json<T>` for a typed body, `Path` for URL segments. If the body doesn't match `CreateLinkRequest`, Axum returns a 400 before your code runs.

▸ Rust vs Python

Python

```python
@app.post('/api/links')
def create(req):
    body = req.json()          # validate yourself
    url = body['url']          # may KeyError
```

Rust

```rust
async fn create_link(State(state): State<Arc<AppState>>, Json(req): Json<CreateLinkRequest>)
    -> Result<(StatusCode, Json<CreateLinkResponse>), AppError>
```

Flask/FastAPI handlers receive a request object you must unpack. Axum unpacks it for you via the parameter types, and the return type declares the success and error shapes — all checked at compile time.

★ The signature is the spec

You can read a handler's first line and know exactly what it consumes and produces. No hidden globals, no untyped request bag.

44 Building a response by hand →

When a tuple isn't enough: the 302 redirect

Most handlers return a tuple or `Json` and let Axum's `IntoResponse` do the work. But the redirect needs a precise `302` with a `Location` header, so we build the `Response` ourselves:

url-shortener/src/api/mod.rs

```rust
async fn redirect(
    State(state): State<Arc<AppState>>,
    Path(code): Path<String>,
) -> Result<Response, AppError> {
    let target = state.service.resolve(code).await?;
    let location = HeaderValue::from_str(target.as_str())
        .map_err(|e| AppError::Internal(Box::new(e)))?;
    let mut response = Response::new(Body::empty());
    *response.status_mut() = StatusCode::FOUND;          // 302
    response.headers_mut().insert(header::LOCATION, location);
    Ok(response)
}
```

Notice every step that could fail is handled: building the header value is a `Result` (a URL could contain an invalid byte), and `?` turns a failure into a clean error. We never assume; the compiler won't let us.

> **⚙ Why 302 and not 301**
>
> A `302` tells browsers "temporary" so they re-request each time — which lets us count every hit. A `301` would be cached and we'd lose the analytics. The type system didn't choose that for us, but it made sure we set the status explicitly and visibly.

> **▶ Try it**

> Change `StatusCode::FOUND` to `StatusCode::MOVED_PERMANENTLY` and re-run the smoke test. What changes in the response? Why might that hurt hit-counting?

45 The builder pattern — readable configuration

Chained methods that assemble a complex value

When something has many optional settings, Rust favors a **builder**: start with a base, chain methods to tweak it, then finish. Opening the database is a builder chain that reads like a checklist:

url-shortener/src/infrastructure/sqlite.rs

```
let options = SqliteConnectOptions::from_str(database_url)?
    .create_if_missing(true)
    .journal_mode(SqliteJournalMode::Wal)
    .synchronous(SqliteSynchronous::Normal)
    .busy_timeout(Duration::from_secs(5));

let pool = SqlitePoolOptions::new()
    .max_connections(max_connections)
    .connect_with(options)
    .await?;
```

Each method returns `Self`, so they chain. The result is self-documenting: create the file if missing, use WAL mode, normal sync, a 5-second busy timeout, a bounded pool. No giant constructor with ten positional arguments.

▸ Rust vs Python

Python

```
engine = create_engine(url,
pool_size=5,
connect_args={...}, ...) #
positional/keyword soup
```

Rust

```
let opts =
SqliteConnectOptions::from_str(url)
    .create_if_missing(true)
    .journal_mode(Wal)
    .busy_timeout(Duration::from_se
```

Python often configures via a long keyword-argument list you must remember. The builder chain names each choice, is checked at compile time, and lets you set only what you need.

▶ Try it

In `sqlite.rs`, find the builder chain. Remove `.busy_timeout(...)` and add a comment explaining what behavior changes when two writers collide. (Hint: see the WAL chapter in the scaling notes.)

46 Dependency injection, concretely

One trait, three implementations, zero coupling

We've met the pieces; here's the payoff in one view. The application depends on the `LinkRepository` trait. `build_app` accepts *any* implementation as `Arc<dyn LinkRepository>`:

url-shortener/src/lib.rs

```
pub fn build_app(config: Config, repo: Arc<dyn LinkRepository>) -> Router
{ /* ... */ }
```

Production injects the real database; tests inject an in-memory fake — **the same `build_app`, the same handlers, the same business logic**:

url-shortener

```
// production (main.rs)
let repo: Arc<dyn LinkRepository> =
    Arc::new(SqliteLinkRepository::connect(&config.database_url,
n).await?);

// tests (tests/links.rs)
let repo: Arc<dyn LinkRepository> =
Arc::new(InMemoryLinkRepository::default());
```

▸ Rust vs Python

Python

```python
class Service:
    def __init__(self,
repo=SqliteRepo()):  # default
hides the dependency
        self.repo = repo
```

Rust

```rust
fn build_app(config: Config,
repo: Arc<dyn LinkRepository>)
-> Router // dependency is
explicit & typed
```

Python DI is informal — pass any object, default to a real one, hope it has the right methods. Rust makes the dependency an explicit, type-checked parameter, so swapping implementations is a one-liner and the compiler verifies the fake matches the real.

✓ What Rust gave us here

Because the seam is a trait, our ~85 tests run without a database, we added a Redis cache as just another implementation, and a future Postgres backend won't touch a line of business logic. That's the whole architecture paying off.

47 Meta — docs, formatting, and living comments

/// doc comments, fmt, and clippy

Comments starting with `///` are **documentation**: tools render them to HTML, and they sit right above the thing they describe. Module-level `//!` comments explain each layer:

url-shortener/src/domain/mod.rs

```
//! Domain layer: pure entities, validation, and the repository port.
//! This module has no dependency on Axum, sqlx, or HTTP.

/// A validated short code: 1-32 ASCII alphanumeric characters.
pub struct ShortCode(String);
```

`cargo fmt` formats everything to one canonical style (no bikeshedding), and `cargo clippy` catches common mistakes and non-idiomatic patterns. Our CI runs both with warnings treated as errors, so the codebase stays clean automatically.

> **★ Docs that can't lie**
>
> Doc comments can contain examples that `cargo test` actually runs, so documentation that drifts from reality fails the build.

48 Appendix — the repo, file by file

Where to look when you want to see a concept in the wild

Use this as a map. Open any file and you'll find the chapters above living in real code.

a guided index of both crates

```
url-shortener/src/
  domain/mod.rs          structs, enums, newtypes, traits, Display, Option,
consts
  application/mod.rs     Result + ?, closures, iterators, loops, generics,
time
  infrastructure/
    memory.rs            Mutex + HashMap, Default, interior mutability
    sqlite.rs            let-else, From/FromStr, error mapping, async trait
impl
  cache.rs               trait + 3 impls (NoOp / InMemory / Redis), Arc<dyn
Cache>
  rate_limit.rs          Instant/Duration, token bucket, per-IP HashMap
  error.rs               error enum, IntoResponse trait impl, match
  config.rs              primitives, parse, iterators, env
  api/mod.rs             async handlers, tuples, generics, middleware, DTOs
(serde)
  lib.rs / main.rs       modules, Arc, #[tokio::main], graceful shutdown
  tests/                 #[test], #[tokio::test], integration tests, the in-
memory double

pastebin-service/        same shape; plus static/app.js (browser AES-256-GCM
client)
```

▶ Final exercise

Pick one chapter, open the file it cites, and run `cargo test` in that crate. Then change one line (e.g. the code length, or a validation rule)

and watch the compiler and tests react. That feedback loop — change, compile, test — is the fastest way to learn Rust.

★ Where to go next

When you're ready for the official deep dives: *The Rust Programming Language* ("the Book"), *Rust by Example* (whose topic order this book followed), and `std` docs. You already have the rarest thing: a real codebase you understand.

49 What Rust gave us — the big picture ★

Putting it together across both apps

You've now seen, in real code, the pillars that make Rust a strong choice for these services. Step back and notice what they bought us:

✓ Safety without a garbage collector

Ownership + borrowing freed memory deterministically — no GC pauses, no leaks, predictable latency under load.

✓ Errors you cannot ignore

`Result`, `Option`, and exhaustive `match` turned whole classes of runtime crashes into compile errors. Our request paths never panic.

✓ Swappable architecture for free

One trait (`LinkRepository` / `PasteRepository`) let us run SQLite in production, an in-memory double in tests, and add a Redis cache — without touching business logic.

✓ Fearless concurrency

A whole toolbox — `Arc` for read-only sharing, a `Mutex` for short critical sections, atomics for lock-free counters, and channels/actors for message passing — let thousands of requests share state with data races ruled out at compile time by `Send` / `Sync`.

> ✓ **One toolchain**
>
> Cargo built, tested, linted, and formatted everything; `#![forbid(unsafe_code)]` guaranteed memory safety on every build.

None of this required you to memorize syntax. It required understanding **why** Rust asks what it asks — and you saw the payoff in two apps you helped build. That's learning by doing. Now go read the source, run `cargo test`, and change something. The compiler is the most patient teacher you'll ever have.